

contrastar tu desempeño posterior

Dato

Respuesta

Nombre completo CARLOS LOENEL CESPEDES ZEBALLOS

Grupo / SOUMFLOUW

Dominio:

Música

¿Qué partes del proyecto desarrollaste o modificaste? Frontend / Backend / BD / JavaScript / APIs / JSON /

R: desarrolle la parte del backend y las base de datos donde se trabajo el proyect006F

Indica archivos, pantallas, endpoints, tablas o funcionalidades

en las que participaste

¿Qué parte conoces menos o no modificaste directamente?

¿Qué parte conoces menos o no modificaste directamente?

Menciona una funcionalidad del proyecto que puedas explicar y modificar hoy

A. Arquitectura general 1.

1. Selección múltiple. ¿Cómo se comunican normalmente el FrontEnd y el BackEnd? b) El FrontEnd realiza peticiones HTTP al BackEnd y recibe datos, generalmente en JSON

2. Desarrollo. Explica qué ocurre desde que el usuario hace clic en “Ver detalle de [película/libro/canción/juego]” hasta que la información aparece en pantalla. Menciona JavaScript, petición HTTP, endpoint/PHP, base de datos, JSON y HTML cuando correspondan

Cuando un usuario hace clic en un botón para ver o reproducir una canción pasa lo siguiente

El usuario hace clic en el botón dentro de la interfaz HTML. JavaScript detecta esta acción mediante un "escuchador de eventos" (addEventListener en el contenedor principal) y captura el ID de la canción o playlist seleccionada.

La función de JavaScript (como `obtenerCancionesPorIds` o `cargarCanciones`) utiliza `fetch` para enviar una petición HTTP de forma asíncrona (sin recargar la página). Esta petición viaja hacia el endpoint del servidor, que en tu caso es el archivo PHP ubicado en la ruta `api/canciones.php`.

Procesamiento y Base de Datos: El script PHP recibe la petición HTTP. Extrae los parámetros (como el ID o el texto de búsqueda) y se conecta a la base de datos (MySQL). Ejecuta una consulta SQL (`SELECT`) para buscar exactamente la información solicitada en las tablas correspondientes.

Respuesta en JSON: La base de datos devuelve los registros a PHP. El servidor toma esta información y la empaqueta en formato JSON (una estructura de texto ligero y organizado), enviándola como respuesta HTTP de vuelta al navegador del usuario.

Renderizado en HTML: Finalmente, JavaScript recibe la respuesta, convierte el JSON en un objeto manejable y utiliza esos datos para generar fragmentos de código HTML dinámico (inyectando los títulos, portadas, artistas, etc.). Estos fragmentos se insertan en el DOM (la estructura de la página), haciendo que la información aparezca en la pantalla.

3. Indica qué tecnologías utilizó tu equipo y explica la función concreta que cumple cada una en TU proyecto: PHP, MySQL/PostgreSQL, JavaScript, JSON, API/Endpoints, Framework/Librería y Postman

Php: Es el motor del BackEnd. Se encarga de recibir las peticiones que hace la página, validar los datos (por ejemplo, cuando se sube una nueva canción en MP3)

Mysql: esta siendo usado como gestor de base de datos en donde se almacenan las canciones que es encuentran en el sitio web

JavaScript: Es el cerebro del FrontEnd. Maneja toda la lógica interactiva: controla el reproductor nativo (`new Audio()`), actualiza la barra de progreso, maneja el volumen, gestiona las playlists en el almacenamiento local (`localStorage`) y realiza llamadas al servidor sin que la página parpadee o se recargue.

JSON: Es el puente de comunicación o formato de intercambio de datos. Permite que PHP envíe listas de canciones complejas y que JavaScript las lea fácilmente para mostrarlas en la interfaz.

API / Endpoints: Son las "puertas de acceso" de tu servidor (ej. api/canciones.php).
Permiten que el FrontEnd se comuniquen con el BackEnd mediante reglas claras,
soportando métodos como GET (para traer música)

B. JavaScript y FrontEnd

4. Analiza: `fetch('api/canciones.php').then(response => response.json()).then(data => mostrarCanciones(data)).catch(error => console.error(error));` Explica qué hace el código y qué papel cumple JavaScript

En esta línea de código, lo que hago es realizar una petición HTTP GET al servidor, específicamente buscando el archivo api/canciones.php.

5. Explica qué hace: `const resultado = datos.filter(item => item.genero === 'Acción');` ¿Cómo aplicarías una operación similar en tu proyecto?

Recorre un arreglo llamado datos y crea un **nuevo arreglo** (resultado) que contendrá únicamente los elementos cuyo atributo genero sea exactamente igual a la palabra 'Acción'

En mi sistema de música, yo utilicé exactamente esta misma lógica funcional. Por ejemplo, se utilizó la función `renderizarBiblioteca()`, donde utilizo `.filter()` para recorrer todo mi `catalogoMusica` y extraer únicamente las canciones que el usuario ha marcado como favoritas (comparando los IDs):

6. Explica: `document.getElementById('titulo').textContent = pelicula.titulo;` ¿qué elemento busca, qué modifica y qué información muestra?

Con esa línea de código, lo que hago es buscar dentro del documento HTML (el DOM) una etiqueta que tenga asignado exactamente el `id="titulo"`

En la construcción de mi reproductor, hago esto todo el tiempo. Por ejemplo, cuando un usuario quiere editar una canción, busco los campos de mi formulario y les inyecto los valores correspondientes usando una lógica muy parecida: `refs.titulo.value = c.titulo`

7. Desarrollo. ¿Qué ocurre en tu proyecto si una petición fetch/AJAX falla?

Explica cómo podría detectarse y cómo se podría informar al usuario

Si en mi proyecto una petición a mi API (api/canciones.php) llega a fallar, ya sea porque se cayó el servidor o no hay internet, la ejecución de mi código es atrapada inmediatamente por el bloque catch (error)

C. Backend / PHP

8. Menciona un archivo PHP real que hayas desarrollado o modificado. Explica qué recibe, qué procesa, qué consulta y qué devuelve

En mi proyecto, desarrollé el archivo que gestiona la conexión a la base de datos, el cual es el cimiento de todo mi BackEnd. Este archivo utiliza PDO (PHP Data Objects) para interactuar con MySQL de forma segura.

A nivel interno, recibe las credenciales y parámetros de configuración del servidor local: el host (127.0.0.1), el nombre de la base de datos (soundflow), el usuario y la contraseña.

9. Selección múltiple. ¿Cuál es la forma adecuada de prevenir inyección SQL?

b) Utilizar sentencias preparadas y parámetros

D. Base de Datos

10. Dibuja o describe dos tablas reales de tu proyecto, indicando claves primarias, claves foráneas y relación

Para mi proyecto "SoundFlow", diseñé una base de datos optimizada con el cotejamiento utf8mb4 para soportar todo tipo de caracteres. Mi arquitectura se centra en una tabla principal muy completa:

Clave Primaria (Primary Key): Es el campo id (INT UNSIGNED AUTO_INCREMENT). Es el identificador único principal.

11. Explica una consulta SQL real de tu proyecto. Explica qué devuelve y dónde se utiliza

```
1. INSERT INTO canciones (external_id, titulo, artista, album, genero, duracion,
imagen_url, audio_url, fuente)
2. VALUES ('local_demo_1', 'Goteo', 'Duki', 'Super Sangre
Joven', 'Urbano', '2:44', 'img/songs/1.jpg', 'audio/cancion1.mp3', 'local')
3. ON DUPLICATE KEY UPDATE
4.     titulo = VALUES(titulo), artista = VALUES(artista), album = VALUES(album);
5.
```

Lo que hace esta consulta es intentar insertar una nueva canción en el catálogo. Pero gracias a la cláusula ON DUPLICATE KEY UPDATE, si la base de datos detecta que ya existe un registro con el mismo external_id (por la regla UNIQUE que le puse a mi tabla), no bloquea el sistema con un error. En su lugar, actualiza automáticamente los campos de esa canción con la nueva información. Retorna el número de filas afectadas (1 si se insertó, 2 si se actualizó).

E. APIs, JSON y Postman

12. Menciona un endpoint real: ruta, método HTTP, parámetros, JSON devuelto y uso en el FrontEnd

Ruta / Endpoint: api/canciones.php

Método HTTP: GET (cuando pido datos).

```
1. {  
2.   "success": true,  
3.   "data": [  
4.     {  
5.       "id": 1,  
6.       "titulo": "Goteo",  
7.       "artista": "Duki",  
8.       "imagen_url": "img/songs/1.jpg",  
9.       "audio_url": "audio/cancion1.mp3"  
10.    }  
11.  ],  
12.  "meta": { "total": 1, "pages": 1 }  
13. }  
14.
```

En mi código JavaScript, la función cargarCanciones() o buscarCanciones() procesa este JSON. Extrae el arreglo que viene dentro de data y se lo pasa a la función renderizarCanciones()

13. Explica la diferencia entre GET y POST y proporciona un ejemplo real de tu proyecto

GET se usa solo para pedir o consultar información. Los datos (como la página o la búsqueda) viajan expuestos directamente en la URL.

Ejemplo en mi proyecto: Lo uso al inicio de la página con `fetch('api/canciones.php?page=1')` para solicitarle a PHP que me traiga la lista de canciones.

POST se usa para enviar nueva información al servidor. Los datos viajan ocultos de forma segura en el cuerpo (body) de la petición HTTP, lo cual es obligatorio al enviar archivos.

Ejemplo en mi proyecto: Lo aplico en mi función guardarCancion(e). Cuando agrego una canción nueva (cuando id no existe), creo un objeto FormData que empaqueta los textos (título, artista) y el archivo de audio (MP3). Luego envío todo esto a api/canciones.php indicando explícitamente method: 'POST' en la petición fetch.

F. Integración

14. Describe el flujo de tu proyecto: HTML → JavaScript → PHP → Base de Datos → PHP → JSON → JavaScript → HTML. Explica qué ocurre en cada etapa

HTML: El usuario escribe el nombre de una canción en el input del buscador en la interfaz.

JavaScript: Un "escuchador" (addEventListener('input')) detecta lo que el usuario escribe. JavaScript captura el texto y hace una petición fetch asíncrona hacia mi servidor.

PHP (Entrada): El archivo api/canciones.php recibe la petición, captura el parámetro de búsqueda y establece la conexión con PDO de forma segura.

Base de Datos (MySQL): PHP ejecuta una consulta SQL preparada (como SELECT * FROM canciones WHERE titulo LIKE ...) en mi base de datos soundflow.

PHP (Salida): MySQL le devuelve los registros encontrados a PHP. PHP los organiza en un arreglo asociativo y utiliza json_encode() para convertir toda esa información a formato JSON.

JSON → JavaScript: El archivo JSON viaja de vuelta por la red. Mi JavaScript recibe la respuesta y la desempaqueta con await respuesta.json(), convirtiéndola nuevamente en un arreglo manejable.

JavaScript → HTML: Finalmente, JavaScript recorre ese arreglo con un método .map(), crea nuevos elementos <article class="card"> con la información de cada canción y reemplaza el contenido del DOM (contenedor.innerHTML), actualizando la pantalla del usuario al instante.

15. Si al hacer clic en un botón no aparecen resultados, indica al menos tres lugares donde investigarías el problema y cómo comprobarías cada uno

Lo primero que haría sería buscar en el código primero donde se encuentra el botón

Gestor de Base de Datos (MySQL): Si el servidor no marca error pero me devuelve un JSON vacío, abriría mi gestor de base de datos (como phpMyAdmin) y ejecutaría la consulta SELECT manualmente.

